7.
```
#include<stdio.h>
#define MAX 100

int adj[MAX][MAX];
int indegree[MAX];
int n;

void topologicalSort(){
    int queue[MAX];
    int front=0, rear=-1;

    int topo[MAX];
    int count=0;

    for(int i=0; i<n; i++){
        indegree[i]=0;
    }
    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            if(adj[i][j]==1){
                indegree[j]++;
            }
        }
    }

    for(int i=0; i<n; i++){
        if(indegree[i]==0){
            queue[++rear]=i;
        }
    }
```

```
while (front <= rear){
    int u = queue[front++];
    topo[count++] = u;

    for (int v = 0; v < n; v++){
        if (adj[u][v] == 1){
            indegree[v]--;

            if (indegree[v] == 0){
                queue[++rear] = v;
            }
        }
    }
}
if (count != n){
    printf("Graph contains a cycle! Topological sorting not
    possible.\n");
}else{
    printf("Topological Order: ");
    for(int i=0; i<n; i++){
        printf("%d", topo[i]);
    }
    printf("\n");
}
}
int main(){
    printf("Enter number of vertices:");
    Scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (int i=0; i<n; i++){
```

```
        for (int j=0; j<n; j++){
            scanf("%d", &adj[i][j]);
        }
    }
    topologicalSort();
    return 0;
}
```

output:

Enter number of vertices: 5
Enter adjacency matrix:

1 0 1 0 1
1 1 1 1 0
0 0 0 0 1
0 0 1 0 1
1 1 0 0 1

Graph contains a cycle! Topological sorting not possible.

Enter number of vertices: 5
Enter adjacency matrix:

0 1 0 0 0
0 0 1 0 0
0 0 0 1 0
0 0 0 0 1
0 0 0 0 0

Topological Order: 0 1 2 3 4

Leetcode Question.

207. Course Schedule.

```
#include<stdio.h>
#include<stdlib.h>
#include<stdbool.h>

bool canFinish(int numCourses, int** prerequisites, int
    prerequisitesSize, int* prerequisitesColSize){
int** graph = (int**)malloc(numCourses*sizeof(int*));
int *graphColSize = (int*)malloc(numCourses, sizeof(int));

for(int i=0; i<numCourses; i++){
    graph[i] = (int*)malloc(numCourses * sizeof(int));
}
int* indegree = (int*)calloc(numCourses, sizeof(int));
for(int i=0; i<prerequestiesSize; i++){
    int a = prerequisities[i][0];
    int b = prerequisities[i][1];

    graph[b][graphColSize[b]++] = a;
    indegree[a]++;
}
int* queue = (int*)malloc(numCourses * sizeof(int));
int front=0, rear=0;
for(int i=0; i<numCourses; i++){
    if(indegree[i]==0){
        queue[rear++] = i;
    }
}
```

```
int count = 0;
while (front < rear ){
    int Course = queue [front++];
    count++;
    for (int i=0; i<graphColSize [course]; i++){
        int neighbour = graph [course][i];
        indegree [neighbour]--;

        if (indegree[neighbour] == 0){
            queue [rear++] = neighbour;
        }
    }
}
return count == numCourses;
```